

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Технології розробки на платформі Java

Лабораторна робота №2

«Збірка проєкту за допомогою Gradle та робота з Java Collections Framework»

Виконав: студент 3 курсу ФІОТ,
групи ІО-41
Вовк Андрій Андрійович
Залікова книжка № 4103

Київ — 2026

1 Тема, мета та завдання

Тема: збірка проєкту за допомогою Gradle та робота з Java Collections Framework.

Мета: навчитися налаштовувати збірку Gradle із Version Catalogs, застосовувати стандартні колекції Java для обробки тексту та перевіряти граничні випадки за допомогою юніт-тестів.

Визначення варіанта: номер залікової книжки $N = 4103$.

$$V = N \bmod 3 = 4103 \bmod 3 = 2.$$

Для варіанта $V = 2$ потрібно створити клас `WordCounter` у пакеті `ua.kpi.comsys.collections`. Метод `countWords(List<String>)` повертає відповідність «слово — кількість входжень», а `getUniqueWords(List<String>)` — множину унікальних слів. Словом вважається послідовність латинських букв і цифр. Слова приводяться до нижнього регістру, порожні фрагменти пропускаються. Якщо список дорівнює `null`, обидва методи повинні кидати `IllegalArgumentException`.

2 Проєктування та реалізація

Для пошуку слів використано регулярний вираз `[A-Za-z0-9]+` і `Matcher.find()`. Збіг нормалізується через `toLowerCase(Locale.ROOT)`, тому результат не залежить від поточної локалі системи. Кількості зберігаються у `HashMap<String, Integer>` та збільшуються за допомогою `Map.merge()`. Унікальні слова повертаються як окрема множина `HashSet<String>`. Порядок елементів у результатах не визначено і завдання його не вимагає.

Метод	**Результат і перевірки**
`countWords()`	Повертає `Map<String, Integer>` і підраховує входження слів у всіх рядках.
`getUniqueWords()`	Повертає `Set<String>` без повторень на основі результату підрахунку.
Обидва методи	Для `null`-списку викидають `IllegalArgumentException`. Також відхиляють `null`-рядок у списку.

Таблиця 1: Поведінка методів класу WordCounter

Залежності JUnit 5 задано в каталозі версій `gradle/libs.versions.toml`. Файл `build.gradle.kts` використовує Java-плагін, Maven Central, JUnit Platform та JaCoCo для перевірки покриття. Проєкт містить Gradle Wrapper. Для реалізації колекцій застосовано лише класи стандартної бібліотеки `java.util`.

3 Перевірка роботи

Тести JUnit 5 перевіряють обидва методи на кількох рядках, нормалізацію регістру, цифри у словах, розділові знаки, порожній вхід, `null` та незалежність повернутих колекцій. Окремий тест змінює системну локаль і перевіряє, що латинська `I` переходить у `i`.

Набір перевірок	**Результат**
`Hello world`, `HELLO, World! hello`	`hello → 3`, `world → 2`
`One, one; TWO`, `two 3`	Множина унікальних слів: `{one, two, 3}`.
Порожні рядки й лише розділь-ники	Результати порожні, порожні слова не з'являються.
Список або його елемент — `null`	`IllegalArgumentException` для обох мето-дів.

Таблиця 2: Приклади та граничні випадки

Команди `./gradlew test` і `./gradlew build` виконано успішно. У звіті Gradle зафіксовано **10 тестів, 0 помилок, 0 пропущених**. JaCoCo показав для `WordCounter` **16 із 16 покритих рядків і 8 із 8 покритих гілок** (100% в обох випадках). Завдання `build` перевіряє цей поріг автоматично; HTML-звіт зберігається в `build/reports/jacoco/test/html/`.

Програму, тести, конфігурацію Gradle та інструкцію із запуску опубліковано у відкритому репозиторії: github.com/coffer-45/lab2-java-collections.

4 Висновок

У роботі налаштовано Java-проєкт із Gradle Wrapper і Version Catalogs та реалізовано частотний аналіз слів стандартними колекціями Java. Додано перевірку некоректних даних і JUnit 5 тести. Збірка успішна; клас `WordCounter` має повне покриття рядків і гілок за результатами JaCoCo.